

SIMATS ENGINEERING

ASSESSMENT TOOL 2

Case study

COURSE NAME: Computer Networks

COURSE CODE: CSA0720

COURSE OUTCOME COVERED

CO3: *Analyze the performance of core network protocols such as TCP, UDP, HTTP, and DNS under varying conditions*

Assessment Tool Weightage: 40%

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

MARK ALLOCATION

	Understanding of Case (25%)	Application of Concepts (25%)	Analysis (20%)	Solution Design (20%)	Clarity (10%)
Q1					
Q2					
Q3					
Q4					
Q5					

1. A multinational IT company uses a cloud-based video conferencing platform for daily client meetings. During peak business hours, employees experience frequent voice distortion, frozen video frames, and delayed communication. Network monitoring reports indicate an 8% packet loss, 180 ms jitter, and fluctuating bandwidth utilization. The IT administrator suspects that the issue is related to the transport protocol or application-level buffering strategy.

Questions

- 1. Analyze whether the problem is primarily caused by the transport protocol (TCP/UDP) or the application layer.**
- 2. Justify your answer based on the communication requirements of real-time multimedia applications.**
- 3. Recommend suitable protocol-level or application-level improvements to enhance conferencing quality.**

Case Study: Performance Issues in Cloud-Based Video Conferencing

Introduction

A multinational IT company uses a cloud-based video conferencing platform for daily client meetings. During peak business hours, employees experience voice distortion, frozen video frames, and delayed communication. Network monitoring shows **8% packet loss, 180 ms jitter, and fluctuating bandwidth utilization**. These conditions significantly affect the quality of real-time audio and video communication. The issue can be analyzed by considering both the **transport layer protocols (TCP/UDP)** and the **application layer buffering and media adaptation mechanisms**.

1. Analysis of the Cause

The problem is primarily related to **transport/network conditions and the way real-time applications handle them**, rather than being caused by the application layer alone.

Real-time video conferencing applications generally prefer **UDP** for transmitting audio and video because UDP provides low latency and does not wait for lost packets to be retransmitted. However, UDP does not provide reliable delivery, congestion control, or packet retransmission. Therefore, the observed **8% packet loss and 180 ms jitter** can directly result in missing audio packets, distorted speech, frozen video frames, and irregular playback.

If TCP were used for real-time media, lost packets would normally be retransmitted. Although this improves reliability, retransmissions can introduce additional delay because later packets may have to wait for missing packets. This phenomenon can be particularly harmful to interactive communication.

Therefore, the major issue is the combination of **high packet loss, high jitter, and fluctuating bandwidth**, with UDP-based real-time transmission being particularly sensitive to these network conditions.

However, the **application layer also contributes** to the problem. If the conferencing application does not use an effective jitter buffer, adaptive bitrate control, packet-loss concealment, and dynamic video-quality adjustment, the effects of network instability become more noticeable to users.

2. Justification Based on Real-Time Multimedia Requirements

Real-time multimedia applications have different requirements from normal web browsing or file downloads.

For applications such as video conferencing, **low delay is more important than perfect reliability**. A small amount of packet loss may be acceptable if it allows audio and video to arrive quickly. Users generally

prefer a short missing portion of audio or a slightly degraded video frame rather than waiting several seconds for retransmitted packets.

Effect of the Observed Network Conditions

8% Packet Loss:

This is relatively high for a video conferencing application. Lost audio packets can cause voice distortion or gaps in speech, while lost video packets can result in frozen or corrupted frames.

180 ms Jitter:

Jitter represents variation in packet arrival time. A jitter level of 180 ms is significant for interactive communication and can cause irregular audio playback, synchronization problems, and delays.

Fluctuating Bandwidth:

When available bandwidth changes frequently, the application may not be able to continuously transmit high-quality audio and video. If the application does not adapt quickly, buffering, frame freezing, or reduced quality may occur.

TCP vs UDP

Feature	TCP	UDP
Reliability	High	Low
Retransmission	Yes	No
Latency	Higher	Lower
Suitable for real-time media	Generally less suitable	Generally more suitable
Effect of packet loss	Retransmits lost packets, increasing delay	Lost packets are not retransmitted
Main advantage	Reliable delivery	Fast, low-latency delivery

For video conferencing, **UDP is generally preferred for real-time media**, because maintaining low latency is more important than guaranteeing delivery of every individual packet.

Therefore, the case study indicates that the observed quality problems are mainly caused by **poor network conditions affecting real-time transport**, while application-level buffering and adaptation determine how effectively the conferencing system can hide or reduce those effects.

3. Recommended Improvements

A. Transport and Network-Level Improvements

1. **Use UDP-based real-time media transport:**
UDP should be preferred for interactive audio and video because it avoids the delays associated with TCP retransmissions.
2. **Use congestion-control mechanisms suitable for real-time media:**
Real-time transport should monitor network conditions and reduce the sending rate when congestion is detected.
3. **Prioritize real-time traffic using QoS:**
Network administrators can configure Quality of Service (QoS) so that voice and video packets receive higher priority than less time-sensitive traffic.
4. **Reduce packet loss:**
The company should identify overloaded network links, wireless interference, insufficient bandwidth, or congested routers that may be responsible for the 8% packet loss.

5. **Improve bandwidth capacity:**

Additional bandwidth or better traffic management should be provided during peak business hours to prevent congestion.

B. Application-Level Improvements

1. **Adaptive bitrate streaming:**

The conferencing application should automatically reduce video resolution and bitrate when available bandwidth decreases and increase quality when network conditions improve.

2. **Jitter buffering:**

A properly designed jitter buffer can temporarily store arriving packets and release them at a more consistent rate, reducing the effects of variable packet arrival times.

3. **Packet-loss concealment:**

For audio, the application can generate an estimated replacement for missing packets instead of producing noticeable gaps or distortion.

4. **Dynamic video-quality adjustment:**

When packet loss becomes high, the application can reduce video resolution or frame rate instead of allowing the video to freeze completely.

5. **Prioritize audio over video:**

During severe network congestion, maintaining clear audio should receive higher priority because communication can continue even if video quality is temporarily reduced.

6. **Monitor network quality continuously:**

The conferencing system should monitor packet loss, jitter, latency, and available bandwidth and automatically adjust its transmission parameters.

2.A multinational software company hosts its web services across multiple continents. Employees frequently report long delays before the application homepage loads. Network analysis shows that the average DNS lookup time is approximately 800 ms, significantly affecting user experience. The organization plans to redesign its DNS infrastructure while ensuring continuous service availability during server failures.

Questions

- 1. Analyze the reasons behind the high DNS resolution time.**
- 2. Propose a suitable DNS architecture using techniques such as Anycast DNS, DNS pre-fetching, and TTL optimization to reduce the lookup time below 50 ms.**
- 3. Evaluate the advantages and limitations of your proposed architecture in terms of performance, scalability, and availability.**

Case Study: DNS Optimization for a Multinational Software Company

Introduction

A multinational software company hosts its web services across multiple continents. Employees frequently experience long delays while loading the application homepage. Network analysis indicates that the **average DNS lookup time is approximately 800 ms**, which is significantly higher than the expected response time for a modern web application.

Since DNS is responsible for converting domain names into IP addresses, slow DNS resolution increases the time required before a user's browser can establish a connection with the web server. The company therefore needs a DNS architecture that provides **low latency, high availability, scalability, and automatic failure handling**.

1. Reasons for the High DNS Resolution Time

The DNS lookup time of 800 ms can be caused by several factors:

a) Geographical Distance

The company operates across multiple continents. If users send DNS queries to a DNS server located far away from them, the request and response have to travel over a longer network path. This increases network latency.

For example, an employee in India contacting a DNS server located in North America may experience significantly higher latency than if the query were handled by a nearby DNS server.

b) Lack of Distributed DNS Servers

If the organization depends on only a few centralized DNS servers, a large number of users may send queries to the same servers. This can result in congestion and increased response time.

c) DNS Cache Misses

When the required DNS record is not available in the local resolver's cache, the resolver must perform additional queries to authoritative DNS servers. This creates multiple network round trips and increases the overall resolution time.

d) Poor TTL Configuration

If DNS records have an unnecessarily short **Time to Live (TTL)**, cached records expire frequently. Resolvers then need to contact authoritative servers repeatedly, increasing DNS traffic and lookup latency. On the other hand, excessively long TTL values can make changes such as server failover slower to propagate. Therefore, TTL must be carefully optimized.

e) DNS Server Overload

During peak working hours, overloaded DNS servers may take longer to process queries. Insufficient server capacity or poor load distribution can contribute to the 800 ms average lookup time.

f) Network Congestion

Packet loss, routing inefficiencies, and congestion between users and DNS servers can increase the time required for DNS queries and responses.

g) Multiple DNS Resolution Steps

DNS resolution may involve several stages, including querying the root server, TLD server, and authoritative DNS server. When caching is unavailable, these additional steps can increase the total lookup time.

2. Proposed DNS Architecture

The company should implement a **globally distributed DNS architecture using Anycast DNS, DNS caching/pre-fetching, and optimized TTL values.**

Proposed Architecture

Users → Nearest Anycast DNS Resolver → Cached DNS Record → Anycast Authoritative DNS → Application Server

The architecture can operate as follows:

1. Deploy multiple DNS servers at geographically distributed locations across **Asia, Europe, North America, and other major regions.**
2. Assign the same Anycast IP address to DNS servers in different locations.
3. Use **BGP routing** to direct users to the nearest or best-performing DNS node.
4. Enable aggressive DNS caching and **DNS pre-fetching** for frequently accessed records.
5. Configure appropriate TTL values so frequently accessed records remain in resolver caches for longer periods.
6. Deploy multiple authoritative DNS servers so that the failure of one server does not interrupt DNS service.
7. Continuously monitor DNS response time, server health, and network performance.

Anycast DNS

Anycast allows multiple DNS servers in different geographical locations to advertise the same IP address. Routing protocols direct a user's DNS request to an appropriate nearby DNS server.

For example:

User in India → Asian DNS node

User in Europe → European DNS node

User in USA → North American DNS node

This reduces the physical network distance and therefore lowers DNS response time.

DNS Pre-Fetching

DNS pre-fetching allows DNS records that are frequently requested or approaching expiration to be refreshed before users need them.

For example, if the company's application domain is accessed frequently, the DNS record can be refreshed in advance. When a user requests the domain, the resolver can return the cached result immediately rather than waiting for a complete DNS resolution process.

This reduces cache-miss delays and improves response time.

TTL Optimization

The company should use a balanced TTL strategy.

For stable DNS records, a moderately high TTL can be used so that records remain cached for longer periods. This reduces the number of DNS queries reaching authoritative servers.

For example, an appropriate TTL could be configured in the range of **5–30 minutes or higher depending on how frequently records change**.

For services requiring rapid failover, a shorter TTL can be used so that changes propagate quickly.

Therefore, TTL should be selected according to the stability and failover requirements of each DNS record rather than using one value for every record.

Achieving Less Than 50 ms

The proposed architecture can significantly reduce DNS lookup time through:

- Geographically distributed Anycast DNS servers
- Local caching
- DNS pre-fetching
- Optimized TTL values
- Load balancing
- Multiple authoritative servers
- Efficient routing
- Continuous server health monitoring

For users whose DNS queries are served from a nearby cached Anycast node, the response can potentially be reduced to **below 50 ms**. However, the exact value depends on the user's network, ISP resolver, geographic location, cache state, and routing conditions. Therefore, the company should measure performance from different regions rather than assuming every query will always be below 50 ms.

3. Advantages and Limitations

Advantages

1. Improved Performance

Anycast directs users toward geographically appropriate DNS infrastructure, reducing network distance and response time. Caching and pre-fetching further reduce the need for repeated DNS resolution.

2. High Availability

Multiple DNS servers can provide redundancy. If one DNS node becomes unavailable, routing can direct queries to another available node.

3. Better Scalability

The DNS workload can be distributed across multiple locations instead of depending on a small number of centralized servers. This allows the infrastructure to support a large number of users across continents.

4. Reduced DNS Traffic

Caching, pre-fetching, and suitable TTL values reduce repeated queries to authoritative DNS servers. This decreases infrastructure load and improves efficiency.

5. Improved Fault Tolerance

Anycast and multiple authoritative servers prevent a single DNS server failure from causing complete service disruption.

Limitations

1. Higher Infrastructure Cost

Deploying DNS infrastructure across multiple continents requires additional servers, networking equipment, monitoring systems, and operational resources.

2. Configuration Complexity

Anycast routing, BGP announcements, DNS caching, TTL management, and health monitoring require experienced network administrators.

3. TTL Trade-Off

A very high TTL improves caching and performance but can delay DNS changes during server failures or migrations. A very low TTL improves change propagation but increases DNS query traffic.

4. Anycast Routing Issues

Anycast depends on Internet routing. Incorrect routing configuration or network instability can cause users to reach an unexpected DNS location.

5. Cache Dependency

DNS pre-fetching and caching are most effective for frequently accessed records. A first-time query or cache miss may still require a complete DNS resolution process.

Performance, Scalability, and Availability Evaluation

Factor	Proposed Architecture	Expected Benefit
Performance	Anycast + caching + pre-fetching	Lower DNS lookup latency
Scalability	Multiple distributed DNS nodes	Handles large global traffic
Availability	Multiple authoritative servers + Anycast	Reduces single points of failure
Traffic Load	Optimized TTL + caching	Reduces repeated DNS queries
Failure Handling	Health monitoring + redundant nodes	Automatic traffic redirection
User Experience	Faster DNS resolution	Faster homepage loading

3. A cloud service provider delivers applications to millions of users worldwide. During promotional events, users experience intermittent delays in accessing cloud-hosted services. Investigation reveals that DNS queries are taking nearly 800 ms because all requests are directed to a single primary DNS server. The company wants a highly available DNS solution capable of handling global traffic efficiently.

Questions

- 1. Examine the impact of centralized DNS architecture on application performance.**
- 2. Design a distributed DNS solution that minimizes lookup latency while maintaining high availability.**
- 3. Analyze how TTL values, Anycast routing, and DNS caching influence system performance and fault tolerance.**

Case Study: Distributed DNS Solution for a Global Cloud Service Provider

Introduction

A cloud service provider delivers applications to millions of users worldwide. During promotional events, the number of users increases significantly, resulting in intermittent delays when accessing cloud-hosted services. Investigation shows that DNS queries are taking nearly **800 ms** because all DNS requests are directed to a **single primary DNS server**.

Since DNS resolution is the first step required before users can connect to the cloud application, the centralized DNS architecture becomes a major performance bottleneck. A distributed DNS solution using **Anycast routing, DNS caching, optimized TTL values, and multiple authoritative DNS servers** can improve performance, scalability, and fault tolerance.

1. Impact of Centralized DNS Architecture on Application Performance

In the existing architecture, all users depend on one primary DNS server for resolving domain names into IP addresses.

a) Increased DNS Lookup Latency

When millions of users send queries to the same DNS server, the server can become overloaded. The processing queue increases, resulting in the observed **800 ms DNS lookup time**.

This delay occurs before the user's browser or application can establish a connection with the cloud service. Therefore, even if the cloud application itself is responding quickly, users may experience a noticeable delay.

b) Single Point of Failure

The primary DNS server represents a **single point of failure**. If it becomes unavailable because of hardware failure, software problems, network issues, or excessive traffic, DNS resolution may stop for users.

As a result, users may be unable to access the cloud application even though the application servers themselves are operating normally.

c) Poor Scalability

A single DNS server cannot efficiently handle sudden increases in global traffic. Promotional events can generate a large number of simultaneous DNS requests, causing CPU, memory, network, and connection-resource exhaustion.

d) Increased Network Distance

Users may be located on different continents, but all DNS queries are sent to the same server. Users who are geographically far from the server may experience additional network latency.

e) Reduced Reliability During Traffic Spikes

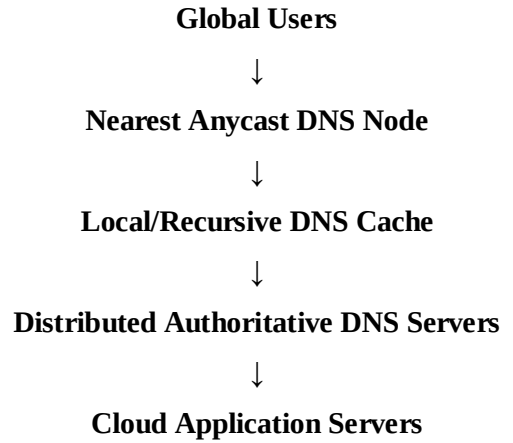
During promotional events, the sudden increase in DNS requests can cause congestion and delayed responses. This results in intermittent application-access problems and poor user experience.

Therefore, the centralized architecture directly contributes to the **800 ms lookup delay and reduced availability**.

2. Proposed Distributed DNS Solution

The cloud provider should replace the single-server architecture with a **globally distributed DNS infrastructure**.

Proposed Architecture



The proposed system should contain multiple DNS nodes distributed across major geographical regions such as **Asia, Europe, North America, and other high-traffic regions**.

Step 1: Deploy Multiple DNS Servers

Instead of using one primary DNS server, the company should deploy multiple authoritative DNS servers across different geographical locations.

For example:

- DNS Server 1 – Asia
- DNS Server 2 – Europe
- DNS Server 3 – North America
- DNS Server 4 – Australia
- Additional backup DNS servers

If one server fails, other DNS servers can continue responding to requests.

Step 2: Implement Anycast Routing

All DNS servers can advertise the same Anycast IP address. Internet routing mechanisms can then direct users toward an appropriate nearby DNS node.

For example:

User in India → Asian DNS node

User in Europe → European DNS node

User in USA → North American DNS node

This reduces network distance and improves DNS response time.

Step 3: Use DNS Caching

DNS responses should be cached by recursive DNS resolvers. When another user requests the same domain, the cached result can be returned without contacting the authoritative DNS server again.

This significantly reduces the number of queries reaching the authoritative infrastructure.

Step 4: Configure Appropriate TTL Values

DNS records should have carefully selected TTL values. Stable records can use longer TTL values so that they remain cached for longer periods.

For services requiring rapid changes or failover, shorter TTL values can be used.

This creates a balance between **performance and fault recovery**.

Step 5: Health Monitoring and Automatic Failover

The DNS infrastructure should continuously monitor server health.

If an Anycast node or authoritative server becomes unavailable, routing and DNS management systems should redirect requests toward healthy servers.

This prevents one server failure from affecting the entire service.

3. Influence of TTL, Anycast Routing, and DNS Caching

A. TTL Values

TTL (Time to Live) specifies how long a DNS record can remain in a resolver's cache.

High TTL

Advantages:

- More cache hits
- Fewer DNS queries
- Lower DNS server workload
- Faster DNS responses

Limitation:

If the IP address changes or a server fails, users may continue receiving the old cached record until the TTL expires.

Low TTL

Advantages:

- DNS changes propagate faster
- Faster response to server changes and failures

Limitations:

- More frequent DNS queries
- Higher DNS server load
- Increased network traffic

Therefore, the cloud provider should use **optimized TTL values based on the stability of each DNS record** rather than using extremely high or extremely low values for everything.

B. Anycast Routing

Anycast allows multiple DNS servers in different locations to use the same IP address.

Performance Impact

Users are generally routed toward a nearby or suitable DNS node, reducing network latency.

For example, instead of every user contacting one server located in a single region, users can be served by geographically distributed nodes.

Fault-Tolerance Impact

If one Anycast location becomes unavailable, routing can direct traffic to another available location.

Therefore, Anycast provides both **lower latency and improved availability**.

C. DNS Caching

DNS caching stores previously resolved DNS records so that repeated requests can be answered without contacting the authoritative DNS infrastructure.

For example, if 100,000 users request the same application domain and the record is already cached by their recursive resolvers, the authoritative DNS servers do not need to process all 100,000 queries.

This:

- Reduces DNS traffic
- Reduces server workload
- Improves response time
- Increases scalability
- Helps the system handle promotional traffic spikes

However, cached records may become outdated if the underlying IP address changes before the cache expires. This is why TTL values must be carefully selected.

Performance and Fault-Tolerance Analysis

Technique	Performance Effect	Fault-Tolerance Effect
Centralized DNS	High latency and server overload	Very low; single point of failure
Distributed DNS	Lower latency	High availability
Anycast Routing	Directs users to suitable nearby DNS nodes	Traffic can move away from failed nodes
DNS Caching	Reduces repeated lookups and server load	Reduces dependency on authoritative servers
High TTL	Faster cache responses	Slower propagation of DNS changes
Low TTL	More frequent DNS resolution	Faster propagation during failures/changes
Multiple DNS Servers	Distributes workload	Prevents complete service failure

4. A financial institution operates an electronic stock trading platform where thousands of buy and sell orders are submitted every second. During market opening hours, the average order acknowledgment latency increases from 1 ms to 40 ms, resulting in delayed trade confirmations and customer dissatisfaction. Network engineers observe increased TCP retransmissions, longer connection establishment times, and excessive buffering.

Questions

- 1. Analyze three possible TCP-related factors responsible for the latency increase, considering flow control, Nagle's algorithm, and SYN queue limitations.**
- 2. Explain how each factor affects transaction processing in high-frequency systems**
- 3. Recommend appropriate TCP configuration changes to reduce latency and improve transaction reliability.**

Case Study: TCP Latency Optimization in an Electronic Stock Trading Platform

Introduction

A financial institution operates an electronic stock trading platform where thousands of buy and sell orders are submitted every second. During normal operation, the average order acknowledgment latency is approximately **1 ms**. However, during market opening hours, this latency increases to **40 ms**, resulting in delayed trade confirmations and customer dissatisfaction.

Network engineers have observed **increased TCP retransmissions, longer connection establishment times, and excessive buffering**. These symptoms indicate that TCP flow control, packet aggregation mechanisms, and connection management may be contributing to the increased latency.

The three important TCP-related factors in this case are **flow control, Nagle's algorithm, and SYN queue limitations**.

1. Three TCP-Related Factors Responsible for Latency Increase

A. TCP Flow Control

TCP uses a **receive window (rwnd)** to control how much data the sender can transmit before receiving acknowledgments from the receiver. This prevents the receiver's buffer from becoming overloaded.

During market opening hours, a large number of orders and acknowledgments are generated simultaneously. If the receiving application cannot process incoming data quickly enough, the receive buffer may become full and the advertised TCP window may become smaller.

This can cause the sender to slow down or temporarily stop transmission.

Effect in This Case

Excessive buffering and a reduced receive window can result in:

- Delayed order transmission

- Increased waiting time for acknowledgments
- Reduced throughput
- Additional buffering
- Increased overall transaction latency

Therefore, inefficient flow-control settings can contribute to the increase from **1 ms to 40 ms**.

B. Nagle's Algorithm

Nagle's algorithm is designed to reduce the number of small TCP packets transmitted over a network. It combines small amounts of data and waits before sending them in some situations.

For normal applications, this can improve network efficiency. However, electronic trading systems frequently transmit **small, time-sensitive messages**, such as order requests and acknowledgments.

If Nagle's algorithm delays a small packet while waiting for an acknowledgment or additional data, the trading application may experience unnecessary latency.

Effect in This Case

For example:

Trading application → Small order message → TCP buffering → Delayed transmission → Exchange/server

Instead of sending the order immediately, TCP may temporarily buffer the small segment. In a high-frequency trading environment, even a small delay can affect transaction processing.

Therefore, Nagle's algorithm can be undesirable for latency-sensitive trading traffic.

C. SYN Queue Limitations

TCP uses the **three-way handshake** to establish a connection:

SYN → SYN-ACK → ACK

During market opening hours, thousands of clients or trading components may attempt to establish connections simultaneously.

The server maintains a queue for partially established connections. If the **SYN backlog/queue** becomes full, new connection attempts may be delayed or dropped.

This causes:

- Longer connection establishment time
- Retransmitted SYN packets
- Increased TCP retransmissions
- Delayed order processing
- Connection failures under extreme load

In a high-frequency trading environment, repeated connection establishment can therefore significantly increase latency.

2. Effect of Each Factor on High-Frequency Transaction Processing

TCP Factor	Effect on System	Impact on Trading
Flow Control	Limits data transmission when receiver buffers are congested	Orders and acknowledgments may be delayed
Nagle's Algorithm	Buffers small TCP segments	Small orders may experience additional transmission delay
SYN Queue Limitation	Limits pending connection requests	Connection establishment becomes slower or fails
TCP Retransmissions	Lost packets are transmitted again	Adds additional delay to transactions
Excessive Buffering	Data waits in queues before transmission/processing	Increases acknowledgment latency

Flow Control in High-Frequency Systems

Trading systems exchange large numbers of small messages within very short time intervals. If the receiving system cannot process data at the same rate as it arrives, TCP flow control reduces the sender's transmission rate.

This creates queues and increases the time required for an order acknowledgment to reach the trader.

Nagle's Algorithm in High-Frequency Systems

Trading orders are generally small but extremely time-sensitive. A delay of even a few milliseconds can affect the order's position and execution opportunity.

If Nagle's algorithm buffers small packets, it can increase transaction latency. Therefore, minimizing unnecessary packet buffering is important for latency-sensitive trading systems.

SYN Queue Limitations in High-Frequency Systems

If the trading platform establishes many short-lived TCP connections during market opening, the SYN queue may become overloaded.

This results in connection establishment delays and retransmissions. If persistent connections are used instead, the number of new TCP handshakes can be reduced significantly.

3. Recommended TCP Configuration Changes

A. Disable Nagle's Algorithm for Latency-Sensitive Connections

The application can enable the `TCP_NODELAY` socket option for trading connections.

This disables Nagle's algorithm and allows small TCP segments to be transmitted without unnecessary waiting.

Benefit:

- Lower message latency
- Faster order transmission
- Faster acknowledgments
- Better responsiveness for small trading messages

This should be applied selectively to latency-critical connections rather than blindly across all network traffic.

B. Optimize TCP Receive and Send Buffers

The operating system's TCP send and receive buffers should be appropriately sized for the application's workload.

The objective is to prevent unnecessary buffering while ensuring that the system can handle traffic bursts.

The company should monitor:

- Receive window size
- Send buffer utilization
- Receive buffer utilization
- TCP queue length
- Packet retransmissions

Proper tuning can prevent excessive buffering and reduce the impact of flow-control limitations.

C. Increase the SYN Backlog

The server's **SYN backlog** should be increased so that it can handle large bursts of incoming connection requests during market opening.

This reduces the probability of:

- SYN queue overflow
- Dropped connection requests
- SYN retransmissions
- Connection establishment delays

The server should also use appropriate TCP SYN-cookie or related protection mechanisms where necessary, while ensuring that security measures do not become a performance bottleneck.

D. Prefer Persistent TCP Connections

The trading platform should avoid repeatedly creating new TCP connections for every order.

Instead, persistent connections can be maintained between trading clients and servers where appropriate.

This eliminates repeated:

SYN → SYN-ACK → ACK

handshakes and significantly reduces connection establishment overhead.

E. Investigate TCP Retransmissions

Since network engineers have already observed increased TCP retransmissions, the institution should identify their cause.

Possible causes include:

- Network congestion
- Packet loss
- Overloaded network devices
- Incorrect network configuration
- Insufficient buffers
- Network interface problems

Reducing packet loss is essential because TCP retransmissions directly increase transaction latency.

F. Use Appropriate TCP Congestion-Control Settings

The operating system should use a congestion-control algorithm suitable for the institution's network environment.

The configuration should be tested under realistic market-opening traffic rather than changed blindly. The goal is to maintain reliability while minimizing unnecessary latency and queue buildup.